py-typedlogic

None

None

None

Table of contents

1. py-typedlogic: Bridging Formal Logic and Typed Python

TypedLogic is a powerful Python package that bridges the gap between formal logic and strongly typed Python code. It allows you to leverage fast logic programming engines like Souffle while specifying your logic in mypy-validated Python code.

links.py run.py stdout Semantics

```
# links.py
from pydantic import BaseModel
from typedlogic import FactMixin, gen2
from typedlogic.decorators import axiom

ID = str

class Link(BaseModel, FactMixin):
    """A link between two entities"""
    source: ID
    target: ID

class Path(BaseModel, FactMixin):
    """An N-hop path between two entities"""
    source: ID
    target: ID
    hops: int

@axiom
def path_from_link(x: ID, y: ID):
    """If there is a link from x to y, there is a path from x to y"""
    assert Link(source=x, target=y) >> Path(source=x, target=y, hops=1)

@axiom
def transitivity(x: ID, y: ID, z: ID, d1: int, d2: int):
    """Transitivity of paths, plus hop counting"""
    assert ((Path(source=x, target=y, hops=d1) & Path(source=y, target=z, hops=d2)) >>
            Path(source=x, target=z, hops=d1+d2))

@axiom
def reflexivity():
    """No paths back to self"""
    assert not any(Path(source=x, target=x, hops=d) for x, d in gen2(ID, int))

from typedlogic.integrations.souffle_solver import SouffleSolver
from links import Link
import links as links

solver = SouffleSolver()
solver.load(links)  ## source for definitions and axioms
# Add data
links = [Link(source='CA', target='OR'), Link(source='OR', target='WA')]
for link in links:
    solver.add(link)
model = solver.model()
for fact in model.iter_retrieve("Path"):
    print(fact)
```

Output:

```
Path(source='CA', target='OR', hops=1)
Path(source='OR', target='WA', hops=1)
Path(source='CA', target='WA', hops=2)
```

To convert the Python code to first-order logic, use the CLI:

`typedlogic convert links.py -t fol`

Output:

```
∀[x:ID y:ID]. Link(x, y) → Path(x, y, 1)
∀[x:ID y:ID z:ID d1:int d2:int]. Path(x, y, d1) ∧ Path(y, z, d2) → Path(x, z, d1+d2)
¬∃[x:ID d:int]. Path(x, x, d)
```

1.1 Key Features

- Write logical axioms and rules using familiar Python syntax

- Benefit from strong typing and mypy validation
- Integration with multiple solvers and logic engines, including Z3 and Souffle
- Compatible with popular Python validation libraries like Pydantic

1.2 Installation

Install TypedLogic using pip:

```
pip install typedlogic
```

With all extras pre-installed:

```
pip install typedlogic[all]
```

You can also use pipx to run the CLI without installing the package globally:

```
pipx run typedlogic --help
```

1.3 Define predicates using Pythonic idioms

Inherit from one of the TypedLogic base models to add semantics to your data model. For Pydantic:

```
from typedlogic.integrations.frameworks.pydantic import FactBaseModel

ID = str

class Link(FactBaseModel):
    source: ID
    target: ID

class Path(FactBaseModel):
    source: ID
    target: ID
```

These can be used in the standard way in Python:

```
links = [Link(source='CA', target='OR'), Link(source='OR', target='WA')]
```

1.4 Specify logical axioms directly in Python

Once you have defined your data model predicates, you can specify logical axioms directly in Python:

```
from typedlogic.decorators import axiom

@axiom
def link_implies_path(x: ID, y: ID):
    """"For all x, y, if there is a link from x to y, then there is a path from x to y"""
    if Link(source=x, target=y):
        assert Path(source=x, target=y)

@axiom
def transitivity(x: ID, y: ID, z: ID):
    """For all x, y, z, if there is a path from x to y and a path from y to z,
       then there is a path from x to z"""
    if Path(source=x, target=y) and Path(source=y, target=z):
        assert Path(source=x, target=z)
```

1.5 Performing reasoning from within Python

Use any of the existing solvers to perform reasoning:

```
from typedlogic.integrations.snakelogic import SnakeSolver

solver = SnakeSolver()
solver.load("links.py")  ## source for definitions and axioms
for link in links:
    solver.add(link)
```

```
model = solver.model()
for fact in model.iter_retrieve("Path"):
    print(fact)
```

outputs:

```
Path(source='CA', target='OR')
Path(source='OR', target='WA')
Path(source='CA', target='WA')
```

2. Gallery

2.1 Examples

This page provides various examples to illustrate the usage of TypedLogic in different scenarios.

2.1.1 Gallery Tutorials

- Synthetic biology design checks: assembly structure, GO-driven pathway feasibility, and GO-CAM curation consistency in one reusable theory.
- SQL goal compiler: bind predicates to SQL tables, compile goals to SQL, run recursive CTEs, and check constraints in SQLite.
- TLog CLI examples: author `.tlog` and literate `.tlog.md` files, convert them, run Clingo, filter materialized predicates, and inspect multiple worlds.
- TLog notebook: notebook form of the same CLI-oriented workflow.

2.1.2 Basic Family Relationships

```
from typedlogic import FactMixin, axiom, gen2
from pydantic import BaseModel

class Parent(BaseModel, FactMixin):
    parent: str
    child: str

class Ancestor(BaseModel, FactMixin):
    ancestor: str
    descendant: str

@axiom
def parent_is_ancestor():
    return all(
        Parent(parent=x, child=y) >> Ancestor(ancestor=x, descendant=y)
        for x, y in gen2(str, str)
    )

@axiom
def ancestor_transitivity():
    return all(
        (Ancestor(ancestor=x, descendant=y) & Ancestor(ancestor=y, descendant=z))
        >> Ancestor(ancestor=x, descendant=z)
        for x, y, z in gen3(str, str, str)
    )

# Usage
theory = Theory(
    name="family_relationships",
    predicate_definitions=[
        PredicateDefinition("Parent", {"parent": str, "child": str}),
        PredicateDefinition("Ancestor", {"ancestor": str, "descendant": str}),
    ],
    sentence_groups=[SentenceGroup(name="axioms", sentences=[parent_is_ancestor(), ancestor_transitivity()])],
)

solver = Z3Solver()
solver.add(theory)
solver.add_fact(Parent(parent="Alice", child="Bob"))
solver.add_fact(Parent(parent="Bob", child="Charlie"))

result = solver.prove(Ancestor(ancestor="Alice", descendant="Charlie"))
print(f"Alice is an ancestor of Charlie: {result}")
```

2.1.3 Type Hierarchies

```
from typedlogic import FactMixin, axiom, gen1
from pydantic import BaseModel

class Animal(BaseModel, FactMixin):
    name: str

class Mammal(Animal):
    pass

class Dog(Mammal):
```

```python
    breed: str

@axiom
def dogs_are_mammals():
    return all(
        Dog(name=x, breed=y) >> Mammal(name=x)
        for x, y in gen2(str, str)
    )

@axiom
def mammals_are_animals():
    return all(
        Mammal(name=x) >> Animal(name=x)
        for x in gen1(str)
    )

# Usage
theory = Theory(
    name="animal_hierarchy",
    predicate_definitions=[
        PredicateDefinition("Animal", {"name": str}),
        PredicateDefinition("Mammal", {"name": str}),
        PredicateDefinition("Dog", {"name": str, "breed": str}),
    ],
    sentence_groups=[SentenceGroup(name="axioms", sentences=[dogs_are_mammals(), mammals_are_animals()])],
)

solver = Z3Solver()
solver.add(theory)
solver.add_fact(Dog(name="Buddy", breed="Labrador"))

result = solver.prove(Animal(name="Buddy"))
print(f"Buddy is an animal: {result}")
```

These examples demonstrate how to use TypedLogic to model relationships and hierarchies, define axioms, and perform logical reasoning. You can expand on these examples to create more complex logical systems tailored to your specific use cases.

2.2 Synthetic Biology Design Checks with TypedLogic

This gallery tutorial turns a small synthetic biology design problem into composable logical checks. The same TypedLogic theory covers assembly structure, GO-driven pathway feasibility, and GO-CAM curation consistency.

The running example is sucrose utilization in *E. coli*: a design needs a transporter (`cscB`) to bring sucrose into the cell and an invertase (`cscA`) to convert intracellular sucrose into glucose and fructose.

2.2.1 Installation

This notebook uses the Clingo solver because the pathway rule is recursive and naturally evaluated as a Datalog-style fixpoint.

```
pip install 'typedlogic[clingo]'
```

2.2.2 Setup

The formal predicates and axioms live in `synbio_theory.py` next to this notebook. Keeping the theory in a normal Python module makes it reusable from tests, scripts, and notebooks.

```
from __future__ import annotations

from contextlib import contextmanager
from pathlib import Path
from typing import Iterable, Sequence
import os
import sys

from IPython.display import Markdown, display
from IPython.lib.display import Code
from typedlogic.integrations.solvers.clingo import ClingoSolver

EXAMPLE_DIR = Path.cwd()
if not (EXAMPLE_DIR / "synbio_theory.py").exists():
    EXAMPLE_DIR = Path.cwd() / "docs" / "examples"

THEORY_PATH = EXAMPLE_DIR / "synbio_theory.py"
sys.path.insert(0, str(EXAMPLE_DIR))

from synbio_theory import (
    AvailableMetabolite,
    CausalEdge,
    CausalRelation,
    DesignContains,
    EncodesProtein,
    FunctionCatalyzes,
    GOAspect,
    GOCAMIndividual,
    HasMolecularFunction,
    Part,
    RequiredMetabolite,
)


@contextmanager
def silence_clingo_stderr():
    """Suppress harmless Clingo messages about predicates that only appear as facts."""
    saved = os.dup(2)
    devnull = os.open(os.devnull, os.O_WRONLY)
    os.dup2(devnull, 2)
    try:
        yield
    finally:
        os.dup2(saved, 2)
        os.close(saved)
        os.close(devnull)


def infer(facts: Iterable[object], *predicate_names: str) -&gt; dict[str, list[tuple[object, ...]]]:
    """Load the synbio theory, add facts, and return selected derived predicates."""
    solver = ClingoSolver()
    solver.load(THEORY_PATH)
    for fact in facts:
        solver.add_fact(fact)

    with silence_clingo_stderr():
        model = solver.model()

    rows = {predicate_name: [] for predicate_name in predicate_names}
    for term in model.ground_terms:
```

```python
        if term.predicate in rows:
            rows[term.predicate].append(term.values)
    return {predicate: sorted(values) for predicate, values in rows.items()}


def markdown_table(headers: Sequence[str], rows: Sequence[Sequence[object]]) -&gt; str:
    """Render a small Markdown table without adding a pandas dependency."""
    lines = ["| " + " | ".join(headers) + " |"]
    lines.append("| " + " | ".join(["---"] * len(headers)) + " |")
    for row in rows:
        lines.append("| " + " | ".join(str(value) for value in row) + " |")
    return "\n".join(lines)


def pairs(values: Sequence[tuple[object, ...]]) -&gt; str:
    """Format binary or assembly-scoped predicate values as compact edges."""
    formatted = []
    for value in values:
        if len(value) == 2:
            left, right = value
            formatted.append(f"{left}-&gt;{right}")
        elif len(value) == 3:
            assembly, left, right = value
            formatted.append(f"{assembly}:{left}-&gt;{right}")
        else:
            formatted.append("-&gt;".join(str(part) for part in value))
    return ", ".join(formatted) or "-"


def csv(values: Sequence[object]) -&gt; str:
    """Format a short sequence for a Markdown table cell."""
    return ", ".join(str(value) for value in values) or "-"
```

```python
Code(filename=str(THEORY_PATH), language="python")
```

```python
"""Synthetic biology design checks for the gallery notebook."""

# ruff: noqa: S101
#
# The theory is intentionally small, but it spans three practical layers:
# 1. Assembly structure checks for compatible Golden Gate-style overhangs.
# 2. GO-driven pathway reachability for engineered metabolic functions.
# 3. GO-CAM curation checks for causal edges between molecular functions.

from dataclasses import dataclass

from typedlogic import Fact, axiom


@dataclass(frozen=True)
class Part(Fact):
    """A DNA part in an ordered assembly."""

    assembly: str
    name: str
    part_type: str
    five_oh: str
    three_oh: str
    position: int


@dataclass(frozen=True)
class CanLigate(Fact):
    """Two parts have compatible overhangs and can ligate."""

    assembly: str
    upstream: str
    downstream: str


@dataclass(frozen=True)
class IntendedAdjacent(Fact):
    """Two parts are adjacent in the intended design order."""

    assembly: str
    upstream: str
    downstream: str


@dataclass(frozen=True)
class IntendedLigationGap(Fact):
    """Two intended adjacent parts have incompatible overhangs."""
```

```
    assembly: str
    upstream: str
    downstream: str


@dataclass(frozen=True)
class MisligationRisk(Fact):
    """A compatible ligation exists outside the intended adjacency order."""

    assembly: str
    upstream: str
    downstream: str


@axiom
def ligation_compatibility(
    assembly1: str,
    n1: str,
    t1: str,
    fo1: str,
    to1: str,
    pos1: int,
    assembly2: str,
    n2: str,
    t2: str,
    fo2: str,
    to2: str,
    pos2: int,
):
    """Derive ligation compatibility when the upstream 3' overhang matches the downstream 5' overhang."""
    if Part(assembly1, n1, t1, fo1, to1, pos1) and Part(assembly2, n2, t2, fo2, to2, pos2):
        if assembly1 == assembly2 and to1 == fo2 and n1 != n2:
            assert CanLigate(assembly1, n1, n2)


@axiom
def intended_adjacency(
    assembly1: str,
    n1: str,
    t1: str,
    fo1: str,
    to1: str,
    pos1: int,
    assembly2: str,
    n2: str,
    t2: str,
    fo2: str,
    to2: str,
    pos2: int,
):
    """Derive intended adjacency from the declared part positions."""
    if Part(assembly1, n1, t1, fo1, to1, pos1) and Part(assembly2, n2, t2, fo2, to2, pos2):
        if assembly1 == assembly2 and pos2 == pos1 + 1:
            assert IntendedAdjacent(assembly1, n1, n2)


@axiom
def intended_ligation_gap(
    assembly1: str,
    n1: str,
    t1: str,
    fo1: str,
    to1: str,
    pos1: int,
    assembly2: str,
    n2: str,
    t2: str,
    fo2: str,
    to2: str,
    pos2: int,
):
    """Flag intended adjacent parts whose overhangs do not match."""
    if Part(assembly1, n1, t1, fo1, to1, pos1) and Part(assembly2, n2, t2, fo2, to2, pos2):
        if assembly1 == assembly2 and pos2 == pos1 + 1 and to1 != fo2:
            assert IntendedLigationGap(assembly1, n1, n2)


@axiom
def misligation_detection(
    assembly: str,
```

```
    upstream: str,
    downstream: str,
    n1: str,
    t1: str,
    fo1: str,
    to1: str,
    pos1: int,
    n2: str,
    t2: str,
    fo2: str,
    to2: str,
    pos2: int,
):
    """Flag compatible ligations that skip over the intended next position."""
    if (
        CanLigate(assembly, upstream, downstream)
        and Part(assembly, n1, t1, fo1, to1, pos1)
        and Part(assembly, n2, t2, fo2, to2, pos2)
        and upstream == n1
        and downstream == n2
    ):
        if pos2 != pos1 + 1:
            assert MisligationRisk(assembly, upstream, downstream)


@dataclass(frozen=True)
class EncodesProtein(Fact):
    """A CDS part encodes a protein."""

    part: str
    protein: str


@dataclass(frozen=True)
class HasMolecularFunction(Fact):
    """A protein has a GO molecular function."""

    protein: str
    go_mf: str


@dataclass(frozen=True)
class FunctionCatalyzes(Fact):
    """A molecular function converts a substrate metabolite to a product metabolite."""

    go_mf: str
    substrate: str
    product: str


@dataclass(frozen=True)
class DesignContains(Fact):
    """A named design contains a part."""

    design: str
    part: str


@dataclass(frozen=True)
class AvailableMetabolite(Fact):
    """A metabolite is available to a design from the environment or an encoded function."""

    design: str
    metabolite: str


@dataclass(frozen=True)
class RequiredMetabolite(Fact):
    """A metabolite that the engineered cell needs for the design intent."""

    design: str
    metabolite: str


@axiom
def metabolite_produced_by_design(
    design: str,
    part: str,
    protein: str,
    go_mf: str,
```

```
    substrate: str,
    product: str,
):
    """Propagate metabolite availability through functions encoded by the design."""
    if (
        DesignContains(design, part)
        and EncodesProtein(part, protein)
        and HasMolecularFunction(protein, go_mf)
        and FunctionCatalyzes(go_mf, substrate, product)
        and AvailableMetabolite(design, substrate)
    ):
        assert AvailableMetabolite(design, product)


@dataclass(frozen=True)
class GOAspect(Fact):
    """The aspect of a GO term: molecular function, biological process, or cellular component."""

    go_term: str
    aspect: str


@dataclass(frozen=True)
class GOCAMIndividual(Fact):
    """An individual in a GO-CAM model instantiating a GO class."""

    iri: str
    go_class: str


@dataclass(frozen=True)
class CausalRelation(Fact):
    """A relation that should connect molecular function individuals in GO-CAM."""

    relation: str


@dataclass(frozen=True)
class CausalEdge(Fact):
    """A causal relation between two GO-CAM individuals."""

    upstream_iri: str
    downstream_iri: str
    relation: str


@dataclass(frozen=True)
class GOCAMViolation(Fact):
    """A derived GO-CAM curation rule violation."""

    individual: str
    rule: str


@axiom
def causal_edge_upstream_must_be_mf(
    up: str,
    down: str,
    rel: str,
    upstream_class: str,
    aspect: str,
):
    """Require the upstream node of a causal edge to instantiate a molecular function."""
    if (
        CausalEdge(up, down, rel)
        and CausalRelation(rel)
        and GOCAMIndividual(up, upstream_class)
        and GOAspect(upstream_class, aspect)
    ):
        if aspect != "MF":
            assert GOCAMViolation(up, "causal_upstream_not_MF")


@axiom
def causal_edge_downstream_must_be_mf(
    up: str,
    down: str,
    rel: str,
    downstream_class: str,
    aspect: str,
```

```
):
    """Require the downstream node of a causal edge to instantiate a molecular function."""
    if (
        CausalEdge(up, down, rel)
        and CausalRelation(rel)
        and GOCAMIndividual(down, downstream_class)
        and GOAspect(downstream_class, aspect)
    ):
        if aspect != "MF":
            assert GOCAMViolation(down, "causal_downstream_not_MF")
```

2.2.3 Layer 1: Assembly Structure

The structural layer treats overhangs as assembly-scoped typed facts. From those facts it derives which parts can ligate, which parts are intended neighbors, whether an intended neighboring pair fails to ligate, and whether any compatible ligation skips the intended next position.

```
safe_assembly = [
    Part("clean", "p_lac", "promoter", "GGAG", "TACT", 0),
    Part("clean", "rbs_b0034", "rbs", "TACT", "AATG", 1),
    Part("clean", "cscB", "cds", "AATG", "GCTT", 2),
    Part("clean", "term_b0015", "terminator", "GCTT", "CGCT", 3),
]

risky_assembly = [
    Part("risky", "p_lac", "promoter", "GGAG", "TACT", 0),
    Part("risky", "rbs_b0034", "rbs", "TACT", "AATG", 1),
    Part("risky", "cscB_skip_rbs", "cds", "TACT", "GCTT", 2),
    Part("risky", "term_b0015", "terminator", "GCTT", "CGCT", 3),
]


def assembly_report(label: str, facts: Sequence[Part]) -&gt; tuple[str, str, str, str, str]:
    results = infer(facts, "CanLigate", "IntendedAdjacent", "IntendedLigationGap", "MisligationRisk")
    gaps = results["IntendedLigationGap"]
    risks = results["MisligationRisk"]
    return (
        label,
        pairs(results["CanLigate"]),
        pairs(results["IntendedAdjacent"]),
        pairs(gaps) if gaps else "none",
        pairs(risks) if risks else "none",
    )


assembly_rows = [
    assembly_report("clean overhang grammar", safe_assembly),
    assembly_report("CDS can skip RBS", risky_assembly),
]

display(
    Markdown(
        markdown_table(
            ["Assembly", "Can ligate", "Intended adjacency", "Intended gap", "Skip risk"],
            assembly_rows,
        )
    )
)
```

Assembly	Can ligate	Intended adjacency	Intended gap	Skip risk
clean overhang grammar	clean:cscB->term_b0015, clean:p_lac->rbs_b0034, clean:rbs_b0034->cscB	clean:cscB->term_b0015, clean:p_lac->rbs_b0034, clean:rbs_b0034->cscB	none	none
CDS can skip RBS	risky:cscB_skip_rbs->term_b0015, risky:p_lac->cscB_skip_rbs, risky:p_lac->rbs_b0034	risky:cscB_skip_rbs->term_b0015, risky:p_lac->rbs_b0034, risky:rbs_b0034->cscB_skip_rbs	risky:rbs_b0034->cscB_skip_rbs	risky:p_lac->cscB_skip_rbs

In the second assembly, `rbs_b0034` and `cscB_skip_rbs` are intended neighbors but have incompatible overhangs. The same assembly also lets `p_lac` ligate directly into `cscB_skip_rbs`, so the theory reports both the intended ligation gap and the skip risk before considering pathway function.

2.2.4 Layer 2: GO-Driven Pathway Feasibility

The functional layer connects parts to proteins, proteins to GO molecular functions, and GO functions to metabolite transformations. A recursive rule derives every metabolite reachable from the growth medium.

```
BIOLOGY = [
    EncodesProtein("cscA", "P10000_cscA"),
    EncodesProtein("cscB", "P30000_cscB"),
    HasMolecularFunction("P10000_cscA", "GO:0004575"),  # sucrose alpha-glucosidase
    HasMolecularFunction("P30000_cscB", "GO:0008515"),  # sucrose:H+ symporter
    FunctionCatalyzes("GO:0008515", "sucrose_out", "sucrose_in"),
    FunctionCatalyzes("GO:0004575", "sucrose_in", "glucose_in"),
    FunctionCatalyzes("GO:0004575", "sucrose_in", "fructose_in"),
]


def pathway_report(label: str, design_parts: Sequence[str]) -&gt; tuple[str, str, str, str]:
    facts = [
        *BIOLOGY,
        *(DesignContains("sucrose_design", part) for part in design_parts),
        AvailableMetabolite("sucrose_design", "sucrose_out"),
        RequiredMetabolite("sucrose_design", "glucose_in"),
    ]
    results = infer(facts, "AvailableMetabolite", "RequiredMetabolite")
    available = sorted({metabolite for _, metabolite in results["AvailableMetabolite"]})
    required = sorted({metabolite for _, metabolite in results["RequiredMetabolite"]})
    missing = [metabolite for metabolite in required if metabolite not in available]
    status = "complete" if not missing else f"gap: {csv(missing)}"
    return label, csv(design_parts), csv(available), status
```

```
pathway_rows = [
    pathway_report("full sucrose pathway", ["cscA", "cscB"]),
    pathway_report("permease only", ["cscB"]),
    pathway_report("invertase only", ["cscA"]),
    pathway_report("empty design", []),
]

display(Markdown(markdown_table(["Design", "Parts", "Reachable metabolites", "Verdict"], pathway_rows)))
```

Design	Parts	Reachable metabolites	Verdict
full sucrose pathway	cscA, cscB	fructose_in, glucose_in, sucrose_in, sucrose_out	complete
permease only	cscB	sucrose_in, sucrose_out	gap: glucose_in
invertase only	cscA	sucrose_out	gap: glucose_in
empty design	-	sucrose_out	gap: glucose_in

The full design reaches intracellular glucose from extracellular sucrose. The permease-only design imports sucrose but cannot cleave it. The invertase-only design has the enzyme, but no route for sucrose to enter the cell.

2.2.5 Layer 3: GO-CAM Curation Consistency

The curation layer checks a common GO-CAM modeling rule: declared causal relations such as `RO:0002413` should connect molecular function individuals, not biological process individuals.

```
GO_ASPECTS = [
    GOAspect("GO:0004575", "MF"),  # sucrose alpha-glucosidase activity
    GOAspect("GO:0008515", "MF"),  # sucrose:H+ symporter activity
    GOAspect("GO:0005992", "BP"),  # trehalose biosynthetic process
    GOAspect("GO:0006012", "BP"),  # galactose metabolic process
]

CAUSAL_RELATIONS = [
    CausalRelation("RO:0002411"),  # causally upstream of
    CausalRelation("RO:0002413"),  # provides direct input for
]


def gocam_report(
    label: str,
    individuals: Sequence[tuple[str, str]],
    edges: Sequence[tuple[str, str, str]],
) -&gt; tuple[str, str, str, str]:
    facts = [
        *GO_ASPECTS,
        *CAUSAL_RELATIONS,
        *(GOCAMIndividual(iri, go_class) for iri, go_class in individuals),
        *(CausalEdge(upstream, downstream, relation) for upstream, downstream, relation in edges),
```

```
    ]
    violations = infer(facts, "GOCAMViolation")["GOCAMViolation"]
    violation_text = ", ".join(f"{iri}: {rule}" for iri, rule in violations) or "none"
    return (
        label,
        csv([f"{iri}={go_class}" for iri, go_class in individuals]),
        pairs([(upstream, downstream) for upstream, downstream, _ in edges]),
        violation_text,
    )
```

```
gocam_rows = [
    gocam_report(
        "valid MF to MF causal edge",
        individuals=[("ind:1", "GO:0008515"), ("ind:2", "GO:0004575")],
        edges=[("ind:1", "ind:2", "RO:0002413")],
    ),
    gocam_report(
        "invalid MF to BP causal edge",
        individuals=[("ind:3", "GO:0008515"), ("ind:4", "GO:0005992")],
        edges=[("ind:3", "ind:4", "RO:0002413")],
    ),
    gocam_report(
        "invalid BP to BP causal edge",
        individuals=[("ind:5", "GO:0006012"), ("ind:6", "GO:0005992")],
        edges=[("ind:5", "ind:6", "RO:0002413")],
    ),
    gocam_report(
        "non-causal relation to BP",
        individuals=[("ind:7", "GO:0008515"), ("ind:8", "GO:0005992")],
        edges=[("ind:7", "ind:8", "BFO:0000050")],
    ),
]

display(Markdown(markdown_table(["GO-CAM model", "Individuals", "Causal edge", "Violations"], gocam_rows)))
```

GO-CAM model	Individuals	Causal edge	Violations
valid MF to MF causal edge	ind:1=GO:0008515, ind:2=GO:0004575	ind:1->ind:2	none
invalid MF to BP causal edge	ind:3=GO:0008515, ind:4=GO:0005992	ind:3->ind:4	ind:4: causal_downstream_not_MF
invalid BP to BP causal edge	ind:5=GO:0006012, ind:6=GO:0005992	ind:5->ind:6	ind:5: causal_upstream_not_MF, ind:6: causal_downstream_not_MF
non-causal relation to BP	ind:7=GO:0008515, ind:8=GO:0005992	ind:7->ind:8	none

The same solver workflow handles design-time checks and curation-time checks. In a production pipeline, the facts could come from a parts registry, GO annotations, Rhea mappings, and GO-CAM RDF exports rather than hand-authored lists.

2.2.6 Inspect the Compiled Rules

TypedLogic parses the Python axioms into a logical theory and the Clingo backend renders that theory as ASP. The recursive pathway rule is the key fixpoint rule.

```
solver = ClingoSolver()
solver.load(THEORY_PATH)
compiled_rules = solver.dump().splitlines()
for rule in compiled_rules:
    if rule.startswith(("availablemetabolite", "intendedligationgap", "misligationrisk", "gocamviolation")):
        print(rule)
```

```
intendedligationgap(Assembly1, N1, N2) :- part(Assembly1, N1, T1, Fo1, To1, Pos1), part(Assembly2, N2, T2, Fo2, To2, Pos2), Assembly1 == Assembly2, Pos2
== Pos1 + 1, To1 != Fo2.
misligationrisk(Assembly, Upstream, Downstream) :- canligate(Assembly, Upstream, Downstream), part(Assembly, N1, T1, Fo1, To1, Pos1), part(Assembly, N2,
T2, Fo2, To2, Pos2), Upstream == N1, Downstream == N2, Pos2 != Pos1 + 1.
availablemetabolite(Design, Product) :- designcontains(Design, Part), encodesprotein(Part, Protein), hasmolecularfunction(Protein, Go_mf),
functioncatalyzes(Go_mf, Substrate, Product), availablemetabolite(Design, Substrate).
gocamviolation(Up, "causal_upstream_not_MF") :- causaledge(Up, Down, Rel), causalrelation(Rel), gocamindividual(Up, Upstream_class),
goaspect(Upstream_class, Aspect), Aspect != "MF".
gocamviolation(Down, "causal_downstream_not_MF") :- causaledge(Up, Down, Rel), causalrelation(Rel), gocamindividual(Down, Downstream_class),
goaspect(Downstream_class, Aspect), Aspect != "MF".
```

2.2.7 Next Steps

This pattern scales by swapping the toy facts for real sources: a parts library for assembly facts, GO and Rhea mappings for function-to-reaction facts, and GO-CAM exports for curation facts. Violations can then be reported back as design review comments or curator feedback.

2.3 SQL Goal Compiler

This notebook demonstrates how to translate a TypedLogic theory plus a query or goal into SQL.

The examples use an in-memory SQLite database so the generated SQL is executed end to end.

2.3.1 Setup

The SQL compiler is a regular TypedLogic compiler. `PredicateBinding` tells the compiler which SQL table and columns back an extensional predicate.

```
import sqlite3

from typedlogic import And, Forall, Implies, NegationAsFailure, Not, PredicateDefinition, SentenceGroup, Term, Theory, Variable
from typedlogic.compilers.sql_compiler import PredicateBinding, SQLCompiler, SQLCompilerConfig
from typedlogic.datamodel import SentenceGroupType


def fetchall(sql: str, setup: list[str] | None = None) -&gt; list[tuple]:
    """Run SQL in a fresh in-memory SQLite database."""
    connection = sqlite3.connect(":memory:")
    try:
        for statement in setup or []:
            connection.execute(statement)
        return list(connection.execute(sql))
    finally:
        connection.close()


compiler = SQLCompiler()
```

2.3.2 Rules As Views

This theory defines `Ancestor` from `Parent`. The recursive rule is compiled as a recursive common table expression.

```
x = Variable("x")
y = Variable("y")
z = Variable("z")

family_theory = Theory(
    predicate_definitions=[
        PredicateDefinition("Parent", {"parent": "str", "child": "str"}),
        PredicateDefinition("Ancestor", {"ancestor": "str", "descendant": "str"}),
    ]
)
family_theory.add(Forall([x, y], Implies(Term("Parent", x, y), Term("Ancestor", x, y))))
family_theory.add(
    Forall(
        [x, y, z],
        Implies(And(Term("Ancestor", x, y), Term("Parent", y, z)), Term("Ancestor", x, z)),
    )
)

ancestor_sql = compiler.compile(
    family_theory,
    goal=Term("Ancestor", "Alice", Variable("who")),
    bindings={"Parent": PredicateBinding("parent_edges", {"parent": "src", "child": "dst"})},
)
print(ancestor_sql)
```

```
fetchall(
    ancestor_sql,
    [
        "CREATE TABLE parent_edges (src TEXT, dst TEXT)",
        "INSERT INTO parent_edges VALUES ('Alice', 'Bob'), ('Bob', 'Carol'), ('Carol', 'Dana')",
    ],
)
```

2.3.3 Inline Unit Clauses

Ground facts do not require table bindings. They become inline CTE branches.

```
person_theory = Theory(predicate_definitions=[PredicateDefinition("Person", {"name": "str"})])
person_theory.add(Term("Person", "Alice"))
person_theory.add(Term("Person", "Bob"))
```

```
person_sql = compiler.compile(person_theory, goal=Term("Person", Variable("name")))
print(person_sql)
fetchall(person_sql)
```

2.3.4 Goal Sentence Groups

If no explicit `goal` is supplied, the compiler uses goal sentence groups from the theory.

```
goal_theory = Theory(predicate_definitions=[PredicateDefinition("Person", {"name": "str"})])
goal_theory.add(Term("Person", "Alice"))
goal_theory.sentence_groups.append(
    SentenceGroup(name="goals", group_type=SentenceGroupType.GOAL, sentences=[Term("Person", Variable("name"))])
)

goal_sql = compiler.compile(goal_theory)
print(goal_sql)
fetchall(goal_sql.removesuffix(";"))
```

2.3.5 Negation As Failure

Negated body atoms are emitted as correlated `NOT EXISTS` subqueries.

```
eligibility_theory = Theory(
    predicate_definitions=[
        PredicateDefinition("Person", {"name": "str"}),
        PredicateDefinition("Blocked", {"name": "str"}),
        PredicateDefinition("Eligible", {"name": "str"}),
    ]
)
eligibility_theory.add(
    Forall([x], Implies(And(Term("Person", x), NegationAsFailure(Term("Blocked", x))), Term("Eligible", x)))
)

eligible_sql = compiler.compile(
    eligibility_theory,
    goal=Term("Eligible", Variable("name")),
    bindings={"Person": "people", "Blocked": "blocked"},
)
print(eligible_sql)
fetchall(
    eligible_sql,
    [
        "CREATE TABLE people (name TEXT)",
        "CREATE TABLE blocked (name TEXT)",
        "INSERT INTO people VALUES ('Alice'), ('Bob')",
        "INSERT INTO blocked VALUES ('Bob')",
    ],
)
```

2.3.6 Constraint Checks

First-order denial constraints compile to validation queries. A returned row is a violation witness.

```
constraint_theory = Theory(
    predicate_definitions=[
        PredicateDefinition("Cat", {"name": "str"}),
        PredicateDefinition("Dog", {"name": "str"}),
    ]
)
constraint_theory.add(Forall([x], Not(And(Term("Cat", x), Term("Dog", x)))))

[constraint_sql] = compiler.compile_constraints(
    constraint_theory,
    config=SQLCompilerConfig(bindings={"Cat": PredicateBinding("cats"), "Dog": PredicateBinding("dogs")}),
)
print(constraint_sql)
fetchall(
    constraint_sql,
    [
        "CREATE TABLE cats (name TEXT)",
        "CREATE TABLE dogs (name TEXT)",
        "INSERT INTO cats VALUES ('Jules'), ('Milo')",
        "INSERT INTO dogs VALUES ('Milo'), ('Rex')",
    ],
)
```

2.4 TLog CLI Examples

TLog is a parser and compiler format, not a separate CLI mode. Use the existing generic commands:

- `convert` for one input theory to one output format
- `dump` for combining and exporting inputs
- `solve` for satisfiability, materialization, and answer-set enumeration
- `test` for quoted `test_case(...)` metadata
- `prove` for quoted goals and lemmas

2.4.1 Convert TLog To Other Formats

```
typedlogic convert docs/examples/tlog/ancestor.tlog -t prolog
```

Example output:

```
%% Predicate Definitions
% parent(parent: PersonID, child: PersonID)
% ancestor(ancestor: PersonID, descendant: PersonID)

parent('Alice', 'Bob').
parent('Bob', 'Charlie').
ancestor(X, Y) :- parent(X, Y).
ancestor(X, Z) :- ancestor(X, Y), ancestor(Y, Z).
```

Other compiler targets work the same way:

```
typedlogic convert docs/examples/tlog/ancestor.tlog -t yaml
typedlogic convert docs/examples/tlog/ancestor.tlog -t fol
typedlogic convert docs/examples/tlog/ancestor.tlog -t souffle
```

2.4.2 Convert To TLog

Use `-t tlog` with any parser-supported input:

```
typedlogic convert docs/examples/tlog/ancestor.tlog -t tlog
```

Example output:

```
type PersonID: str.

pred parent(parent: PersonID, child: PersonID).
pred ancestor(ancestor: PersonID, descendant: PersonID).

parent('Alice', 'Bob').
parent('Bob', 'Charlie').
/// Direct parent links are ancestor links.
all x, y | ancestor(x, y) :- parent(x, y).
/// Ancestor links are transitive.
all x, y, z | ancestor(x, z) :- (ancestor(x, y) & ancestor(y, z)).
```

2.4.3 Literate Markdown

Files ending in `.tlog.md` are parsed as Markdown wrappers around fenced TLog blocks:

```
typedlogic dump docs/examples/tlog/literate-rules.tlog.md -t prolog
```

The prose is ignored. Fenced `tlog`, `typedlogic`, or `logic` blocks are parsed in order.

2.4.4 Run Inference With Clingo

```
typedlogic solve docs/examples/tlog/ancestor.tlog \
  --solver clingo \
```

```
  --show ancestor \
  --max-models 1
```

Example output:

```
Satisfiable: True

=== Model 1 ===
ancestor(Alice, Bob)
ancestor(Bob, Charlie)
ancestor(Alice, Charlie)

Total models shown: 1
```

`--show ancestor` filters materialized output to selected predicates. It is a generic `solve` option and works for other syntaxes too.

To inspect the generated solver-specific program before solving, use `--dump-program`:

```
typedlogic solve docs/examples/tlog/ancestor.tlog \
  --solver clingo \
  --dump-program
```

2.4.5 Show Multiple Worlds

Answer-set solvers such as Clingo can produce multiple models:

```
typedlogic solve docs/examples/tlog/worlds.tlog \
  --solver clingo \
  --show selected \
  --max-models 2
```

Example output:

```
Satisfiable: True

=== Model 1 ===
selected(coffee)

=== Model 2 ===
selected(tea)

Total models shown: 2
```

The input syntax is still just TLog; the multiple-world behavior comes from the chosen solver.

2.4.6 Run Quoted Tests

TLog files can include test cases as quoted metadata:

```
pred human(name: str).
pred mortal(name: str).

mortal(x) :- human(x).

test_case(
  "socrates_mortality",
  given(that(human("socrates"))),
  expect(that(satisfiable() & mortal("socrates") & not philosopher("socrates")))
).
```

`solve` ignores these test cases. Run them explicitly with `test`:

```
typedlogic test docs/examples/tlog/mortality.tlog --solver clingo
typedlogic test docs/examples/tlog/mortality.tlog --solver clingo --test socrates_mortality
```

Example output:

```
PASS socrates_mortality
1 test case(s), 0 failed, 0 unknown
```

Use `--dump-program` to print the generated solver program for each test fixture before expectations are checked.

2.4.7 Prove Lemmas

Lemmas are quoted proof obligations, not axioms:

```
lemma("socrates_is_mortal", that(mortal("socrates"))).
```

Run proof obligations explicitly:

```
typedlogic prove docs/examples/tlog/mortality.tlog --solver z3
typedlogic prove docs/examples/tlog/mortality.tlog --solver z3 --target lemmas
typedlogic prove docs/examples/tlog/mortality.tlog --solver z3 --name socrates_is_mortal
```

Example output:

```
PASS lemma socrates_is_mortal: mortal('socrates')
1 obligation(s), 0 failed, 0 unknown
```

2.5 TLog CLI Workflow

This notebook shows the command-line workflow for TLog. TLog is just another parser/compiler syntax: the generic `typedlogic convert`, `typedlogic dump`, and `typedlogic solve` commands do the work.

```
from pathlib import Path

workspace = Path("tlog-demo")
workspace.mkdir(exist_ok=True)
ancestor_tlog = workspace / "ancestor.tlog"
ancestor_tlog.write_text(
    """
type PersonID: str.

pred parent(parent: PersonID, child: PersonID).
pred ancestor(ancestor: PersonID, descendant: PersonID).

parent("Alice", "Bob").
parent("Bob", "Charlie").

/// Direct parent links are ancestor links.
ancestor(x, y) :- parent(x, y).

/// Ancestor links are transitive.
ancestor(x, z) :- ancestor(x, y), ancestor(y, z).
    """.strip()
    + "\n",
    encoding="utf-8",
)
ancestor_tlog
```

2.5.1 Convert TLog To Other Formats

The input format is inferred from `.tlog`; the output format is selected with `-t`.

```
!typedlogic convert tlog-demo/ancestor.tlog -t prolog
```

```
!typedlogic convert tlog-demo/ancestor.tlog -t yaml
```

2.5.2 Convert Back To TLog

`tlog` is also a compiler target.

```
!typedlogic convert tlog-demo/ancestor.tlog -t tlog
```

2.5.3 Run Inference With Clingo

`--show` filters materialized ground terms by predicate. `--max-models` limits model output.

```
!typedlogic solve tlog-demo/ancestor.tlog --solver clingo --show ancestor --max-models 1
```

2.5.4 Literate Markdown

A `.tlog.md` file can mix prose and fenced TLog blocks.

```
literate = workspace / "literate-rules.tlog.md"
literate.write_text(
    """
# Family Rules

Only fenced `tlog` blocks are parsed.

```tlog
pred parent(parent: str, child: str).
pred ancestor(ancestor: str, descendant: str).
parent("Alice", "Bob").
∀ x, y | parent(x, y) -> ancestor(x, y).
```

""".strip() + "\n", encoding="utf-8",) literate
```

```
</div>
</div>
<div class="cell border-box-sizing code_cell rendered" markdown="1">
<div class="input">

```python
!typedlogic dump tlog-demo/literate-rules.tlog.md -t prolog
```

2.5.5 Multiple Worlds

When the selected solver supports multiple models, the same generic `solve` command can show them.

```
worlds_tlog = workspace / "worlds.tlog"
worlds_tlog.write_text(
    """
pred selected(option: str).
pred hidden(option: str).

selected("tea") | selected("coffee").
hidden("noise").
    """.strip()
    + "\n",
    encoding="utf-8",
)
worlds_tlog
```

```
!typedlogic solve tlog-demo/worlds.tlog --solver clingo --show selected --max-models 2
```
```

3. Concepts

3.1 typed-logic: Bridging Formal Logic and Typed Python

typed-logic is a Python package that allows Python data models to be augmented using formal logical statements, which are then interpreted by *solvers* which can reason over combinations of programs and data allowing for *satisfiability checking*, and the generation of new data.

Currently the solvers supported are:

- Z3
- Souffle
- Clingo
- Prover9
- Snakelog
- ProbLog

With support for more solvers (Vampire, OWL reasoners, etc.) planned.

typed-logic is aimed primarily at software developers and data modelers who are logic-curious, but don't necessarily have a background in formal logic. It is especially aimed at Python developers who like to use lightweight ways of ensuring program and data correctness, such as Pydantic for data and mypy for type checking of programs.

3.1.1 Key Features

- Write logical axioms and rules using familiar Python syntax
- Benefit from strong typing and mypy validation
- Seamless integration with logic programming engines
- Support for various solvers, including Z3 and Souffle
- Compatible with popular Python libraries like Pydantic

3.1.2 Why TypedLogic?

TypedLogic combines the best of both worlds: the expressiveness and familiarity of Python with the power of formal logic and fast logic programming engines. This unique approach allows developers to:

1. Write more maintainable and less error-prone logical rules
2. Catch type-related errors early in the development process
3. Seamlessly integrate logical reasoning into existing Python projects
4. Leverage the performance of specialized logic engines without sacrificing the Python ecosystem

Get started with TypedLogic and experience a new way of combining logic programming with strongly typed Python!

3.2 Core Concepts

TypedLogic is built around several core concepts that blend logical programming with typed Python. Understanding these concepts is crucial for effectively using the library.

3.2.1 Use Python idioms to define your data structures

Facts are the basic units of information in TypedLogic. They are represented as Python classes that inherit from both `pydantic.BaseModel` and `FactMixin`. This approach allows you to define strongly-typed facts with automatic validation.

Example:

```
from pydantic import BaseModel
from typedlogic import FactMixin

PersonID = str
PetID = str

class Person(BaseModel, FactMixin):
    name: PersonID

class PersonAge(BaseModel, FactMixin):
    name: PersonID
    age: int

class Pet(BaseModel, FactMixin):
    name: PetID

class PetSpecies(BaseModel, FactMixin):
    name: PetID
    species: str

class OwnsPet(BaseModel, FactMixin):
    person: PersonID
    pet: PetID
```

Note in many logic frameworks these definitions don't have a direct translation, but in sorted logics, these may correspond to predicate definitions.

3.2.2 Axioms

Axioms are logical rules or statements that define relationships between facts. In TypedLogic, axioms are defined using Python functions decorated with `@axiom`.

Example:

```
from typedlogic.decorators import axiom

@axiom
def constraints(person: PersonID, pet: PetID):
    if OwnsPet(person=person, pet=pet):
        assert Person(name=person) and Pet(name=pet)
```

Note that the Python code in the function definition is **never executed**. The code must be in a subset of python that is translated to logic statements.

You can also derive new facts from axioms:

```
class SameOwner(BaseModel, FactMixin):
    pet1: PetID
    pet2: PetID

from typedlogic.decorators import axiom

@axiom
def entail_same_owner(person: PersonID, pet1: PetID, pet2: PetID):
    if OwnsPet(person=person, pet=pet1) and OwnsPet(person=person, pet=pet2):
        assert SameOwner(pet1=pet1, pet2=pet2)
```

3.2.3 Generators

Generators like `gen1`, `gen2`, etc., are used within axioms to create typed placeholders for variables. They help maintain type safety while defining logical rules.

You can use generators in combination with Python `all` and `any` functions to express quantified sentences:

```
@axiom
def entail_same_owner():
    assert all(SameOwner(pet1=pet1, pet2=pet2)
```

```
                for person, pet1, pet2 in gen3(PersonID, PetID, PetID)
                if OwnsPet(person=person, pet=pet1) and OwnsPet(person=person, pet=pet2))
```

See Generators for more information.

3.2.4 Solvers

TypedLogic supports multiple solvers, including Z3 and Souffle. Solvers are responsible for reasoning over the facts and axioms to derive new information or check for consistency.

Example (using the Z3 solver):

```
from typedlogic.integrations.solvers.z3 import Z3Solver

solver = Z3Solver()
solver.add(theory)
result = solver.check()
```

3.2.5 Theories

A Theory in TypedLogic is a collection of predicate definitions, facts, and axioms. It represents a complete knowledge base that can be reasoned over.

Example:

```
from typedlogic import Theory

theory = Theory(
    name="family_relationships",
    predicate_definitions=[...],
    sentence_groups=[...],
)
```

3.3 Data Model

Data model for the typed-logic framework.

Overview

This module defines the core classes and structures used to represent logical constructs such as sentences, terms, predicates, and theories. It is based on the Common Logic Interchange Format (CLIF) and the Common Logic Standard (CL), with additions to make working with simple type systems easier.

Logical axioms are called sentences which organized into theories., which can be loaded into a solver.

While one of the goals of typed-logic is to be able to write logic intuitively in Python, this data model is independent of the mapping from the Python language to the logic language; it can be used independently of the python syntax.

Here is an example:

```
>>> from typedlogic import Term, Forall, Implies
>>> x = Variable('x')
>>> y = Variable('y')
>>> pdef = PredicateDefinition(predicate='FriendOf',
...                           arguments={'x': 'str', 'y': 'str'}),
>>> theory = Theory(
...     name="My theory",
...     predicate_definitions=[pdef],
... )
>>> s = Forall([x, y],
...            Implies(Term('friend_of', x, y),
...                    Term('friend_of', y, x)))
>>> theory.add(s)
```

Classes

3.3.1 `PredicateDefinition` `dataclass`

Defines the name and arguments of a predicate.

Example:

```
>>> pdef = PredicateDefinition(predicate='FriendOf',
...                           arguments={'x': 'str', 'y': 'str'})
```

The arguments are mappings between variable names and types. You can use either base types (e.g. 'str', 'int', 'float') or custom types.

Custom types should be defined in the theory's `type_definitions` attribute.

```
>>> pdef = PredicateDefinition(predicate='FriendOf',
...                           arguments={'x': 'Person', 'y': 'Person'})
>>> theory = Theory(
...     name="My theory",
...     type_definitions={'Person': 'str'},
...     predicate_definitions=[pdef],
... )
```

Model:

```
classDiagram
class PredicateDefinition {
    +String predicate
    +Dict arguments
    +String description
    +Dict metadata
}
PredicateDefinition --> "*" PredicateDefinition : parents
```

Source code in `src/typedlogic/datamodel.py`

```python
@dataclass
class PredicateDefinition:
    """
    Defines the name and arguments of a predicate.

    Example:

        >>> pdef = PredicateDefinition(predicate='FriendOf',
        ...                            arguments={'x': 'str', 'y': 'str'})

    The arguments are mappings between variable names and types.
    You can use either base types (e.g. 'str', 'int', 'float') or custom types.

    Custom types should be defined in the theory's `type_definitions` attribute.

        >>> pdef = PredicateDefinition(predicate='FriendOf',
        ...                            arguments={'x': 'Person', 'y': 'Person'})
        >>> theory = Theory(
        ...     name="My theory",
        ...     type_definitions={'Person': 'str'},
        ...     predicate_definitions=[pdef],
        ... )

    Model:

    ```mermaid
    classDiagram
    class PredicateDefinition {
        +String predicate
        +Dict arguments
        +String description
        +Dict metadata
    }
    PredicateDefinition --> "*" PredicateDefinition : parents
    ```

    """

    predicate: str
    arguments: Dict[str, str]
    description: Optional[str] = None
    metadata: Optional[Dict[str, Any]] = None
    parents: Optional[List[str]] = None
    python_class: Optional[Type] = None

    def argument_base_type(self, arg: str) -> str:
        typ = self.arguments[arg]
        try:
            import pydantic

            if isinstance(typ, pydantic.fields.FieldInfo):
                typ = typ.annotation
        except ImportError:
            pass
        return str(typ)

    @classmethod
    def from_class(cls, python_class: Type) -> "PredicateDefinition":
        """
        Create a predicate definition from a python class

        :param predicate_class:
        :return:
        """
        return PredicateDefinition(
            predicate=python_class.__name__,
            arguments={k: v for k, v in python_class.__annotations__.items()},
)
```

`from_class(python_class)` `classmethod`

Create a predicate definition from a python class

Parameters:

Name	Type	Description	Default
`predicate_class`			*required*

Returns:

Type	Description
`PredicateDefinition`	

Source code in `src/typedlogic/datamodel.py`

```
@classmethod
def from_class(cls, python_class: Type) -> "PredicateDefinition":
    """
    Create a predicate definition from a python class

    :param predicate_class:
    :return:
    """
    return PredicateDefinition(
        predicate=python_class.__name__,
        arguments={k: v for k, v in python_class.__annotations__.items()},
    )
```

3.3.2 `Variable` `dataclass`

A variable in a logical sentence.

```
>>> x = Variable('x')
>>> y = Variable('y')
>>> s = Forall([x, y],
...            Implies(Term('friend_of', x, y),
...                    Term('friend_of', y, x)))
```

Variables can have domains (types) specified:

```
>>> x = Variable('x', domain='str')
>>> y = Variable('y', domain='str')
>>> z = Variable('y', domain='int')
>>> xa = Variable('xa', domain='int')
>>> ya = Variable('ya', domain='int')
>>> s = Forall([x, y, z],
...            Implies(And(Term('ParentOf', x, y),
...                        Term('Age', x, xa),
...                        Term('Age', y, ya)),
...                    Term('OlderThan', x, y)))

The domains should be either base types or defined types in the theory's `type_definitions` attribute.
```

Source code in `src/typedlogic/datamodel.py`

```
@dataclass
class Variable:
    """
    A variable in a logical sentence.

        >>> x = Variable('x')
        >>> y = Variable('y')
        >>> s = Forall([x, y],
        ...            Implies(Term('friend_of', x, y),
        ...                    Term('friend_of', y, x)))

    Variables can have domains (types) specified:

        >>> x = Variable('x', domain='str')
        >>> y = Variable('y', domain='str')
        >>> z = Variable('y', domain='int')
        >>> xa = Variable('xa', domain='int')
        >>> ya = Variable('ya', domain='int')
        >>> s = Forall([x, y, z],
        ...            Implies(And(Term('ParentOf', x, y),
        ...                        Term('Age', x, xa),
        ...                        Term('Age', y, ya)),
        ...                    Term('OlderThan', x, y)))

        The domains should be either base types or defined types in the theory's `type_definitions` attribute.

    """

    name: str
    domain: Optional[str] = None
    constraints: Optional[List[str]] = None

    def __eq__(self, other):
        return isinstance(other, Variable) and self.name == other.name

    def __str__(self):
        return "?" + self.name

    def __hash__(self):
        return hash(self.name)

    def as_sexpr(self) -> SExpression:
        sexpr = [type(self).__name__, self.name]
        if self.domain:
            return sexpr + [self.domain]
        else:
            return sexpr

    @staticmethod
    def create(names: str) -> tuple['Variable', ...]:
        return tuple(Variable(name.strip()) for name in names.split())
```

3.3.3 `Sentence`

Bases: `ABC`

Base class for logical sentences.

Do not use this class directly; use one of the subclasses instead.

Model:

```
classDiagram
Sentence <|-- Term
Sentence <|-- BooleanSentence
Sentence <|-- QuantifiedSentence
Sentence <|-- Extension
```

Source code in `src/typedlogic/datamodel.py`

```
class Sentence(ABC):
    """
    Base class for logical sentences.

    Do not use this class directly; use one of the subclasses instead.

    Model:

    ```mermaid
    classDiagram
    Sentence <|-- Term
    Sentence <|-- BooleanSentence
    Sentence <|-- QuantifiedSentence
    Sentence <|-- Extension
    ```


    """

    def __init__(self):
        self._annotations = {}

    def __and__(self, other):
        return And(self, other)

    def __or__(self, other):
        return Or(self, other)

    def __invert__(self):
        return Not(self)

    def __sub__(self):
        return NegationAsFailure(self)

    def __rshift__(self, other):
        return Implies(self, other)

    def __lshift__(self, other):
        return Implied(self, other)

    def __xor__(self, other):
        return Xor(self, other)

    def iff(self, other):
        return Iff(self, other)

    def __lt__(self, other):
        return Term(operator.lt.__name__, self, other)

    def __le__(self, other):
        return Term(operator.le.__name__, self, other)

    def __gt__(self, other):
        return Term(operator.gt.__name__, self, other)

    def __ge__(self, other):
        return Term(operator.ge.__name__, self, other)

    def __add__(self, other):
        return Term(operator.add.__name__, self, other)

    def __rmul__(self, other):
        # TODO: avoid circular import
        from typedlogic.extensions.probabilistic import Probability, That
        return Probability(self, That(other))

    @property
    def annotations(self) -> Dict[str, Any]:
        """
        Annotations for the sentence.

        Annotations are always logically silent, but can be used to store metadata or other information.

        :return:
        """
        return self._annotations or {}

    def add_annotation(self, key: str, value: Any):
        """
        Add an annotation to the sentence

        :param key:
        :param value:
        :return:
        """
        if not self._annotations:
            self._annotations = {}
        self._annotations[key] = value

    def as_sexpr(self) -> SExpression:
        raise NotImplementedError(f"type = {type(self)} // {self}")

    @property
    def arguments(self) -> List[Any]:
        raise NotImplementedError(f"type = {type(self)} // {self}")
```

`annotations` `property`

Annotations for the sentence.

Annotations are always logically silent, but can be used to store metadata or other information.

Returns:

Type	Description
`Dict[str, Any]`	

`add_annotation(key, value)`

Add an annotation to the sentence

Parameters:

Name	Type	Description	Default
`key`	`str`		*required*
`value`	`Any`		*required*

Returns:

Type	Description

Source code in `src/typedlogic/datamodel.py`

```
def add_annotation(self, key: str, value: Any):
    """
    Add an annotation to the sentence

    :param key:
    :param value:
    :return:
    """
    if not self._annotations:
        self._annotations = {}
    self._annotations[key] = value
```

3.3.4 `Term`

Bases: `Sentence`

An atomic part of a sentence.

A ground term is a term with no variables:

```
>>> t = Term('FriendOf', 'Alice', 'Bob')
>>> t
FriendOf(Alice, Bob)
>>> t.values
('Alice', 'Bob')
```

```
>>> t.is_ground
True
```

Keyword argument based initialization is also supported:

```
>>> t = Term('FriendOf', dict(about='Alice', friend='Bob'))
>>> t.values
('Alice', 'Bob')
>>> t.positional
False
```

Mappings:

- Corresponds to AtomicSentence in Common Logic

Source code in `src/typedlogic/datamodel.py`

```python
class Term(Sentence):
    """
    An atomic part of a sentence.

    A ground term is a term with no variables:

        >>> t = Term('FriendOf', 'Alice', 'Bob')
        >>> t
        FriendOf(Alice, Bob)
        >>> t.values
        ('Alice', 'Bob')
        >>> t.is_ground
        True

    Keyword argument based initialization is also supported:

        >>> t = Term('FriendOf', dict(about='Alice', friend='Bob'))
        >>> t.values
        ('Alice', 'Bob')
        >>> t.positional
        False

    Mappings:

     - Corresponds to AtomicSentence in Common Logic
    """

    def __init__(self, predicate: str, *args, **kwargs):
        self.predicate = predicate
        if not args:
            self.positional = None
            bindings = {}
        elif len(args) == 1 and isinstance(args[0], dict):
            bindings = args[0]
            self.positional = False
        else:
            bindings = {f"arg{i}": arg for i, arg in enumerate(args)}
            self.positional = True
        self.bindings = bindings
        self._annotations = kwargs

    @property
    def is_constant(self):
        """
        :return: True if the term is a constant (zero arguments)
        """
        return not self.bindings

    @property
    def is_ground(self):
        """
        :return: True if none of the arguments are variables
        """
        return not any(isinstance(v, Variable) for v in self.bindings.values())

    @property
    def values(self) -> Tuple[Any, ...]:
        """
        Representation of the arguments of the term as a fixed-position tuples
        :return:
        """
        return tuple(self.bindings.values())

    @property
    def variables(self) -> List[Variable]:
        """
        :return: All of the arguments that are variables
        """
        return [v for v in self.bindings.values() if isinstance(v, Variable)]

    @property
    def variable_names(self) -> List[str]:
        return [v.name for v in self.bindings.values() if isinstance(v, Variable)]

    def make_keyword_indexed(self, keywords: List[str]):
        """
        Convert positional arguments to keyword arguments
        """
        if self.positional:
            self.bindings = {k: v for k, v in zip(keywords, self.bindings.values(), strict=False)}
            self.positional = False

    def __repr__(self):
        if not self.bindings:
            return f"{self.predicate}"
        elif self.positional:
            return f'{self.predicate}({", ".join(f"{v}" for v in self.bindings.values())})'
        else:
            return f'{self.predicate}({", ".join(f"{v}" for k, v in self.bindings.items())})'

    def __eq__(self, other):
        # return isinstance(other, Term) and self.predicate == other.predicate and self.bindings == other.bindings
        return isinstance(other, Term) and self.predicate == other.predicate and self.values == other.values

    def __hash__(self):
        return hash((self.predicate, tuple(self.values)))

    def as_sexpr(self) -> SExpression:
        return [self.predicate] + [as_sexpr(v) for v in self.bindings.values()]
```